

Relazione Tecnica di Progetto: Motore RAG Ibrido On-Premise

Autore: Marco Troiano

Destinatario: NetCom Group S.p.a.

Progetto: Sviluppo di sistemi basati su Agenti AI per l'interrogazione semantica di dataset letterari: architetture, implementazione e profili di sicurezza.

Data: 11 Settembre 2026

1. Executive Summary

Il presente documento illustra l'architettura definitiva e le modalità di esecuzione del sistema RAG (Retrieval-Augmented Generation) sviluppato durante il periodo di tirocinio. Il sistema è stato progettato per interrogare archivi storici e biblioteconomici attraverso il linguaggio naturale, garantendo tre pilastri aziendali fondamentali: **Data Sovereignty (Privacy by Design)**, **Affidabilità dell'Output (Fault Tolerance)** e **Ottimizzazione delle Risorse (Caching e Ibridazione)**.

L'infrastruttura codice è attualmente in stato di *Code Freeze* per permettere la stesura dell'elaborato di tesi, avendo raggiunto e superato tutti gli obiettivi fissati per l'MVP (Minimum Viable Product).

2. Highlight Architetture e Valore di Business

Il sistema si discosta dai classici tutorial accademici per implementare pattern di livello Enterprise:

- **Ricerca Ibrida Nativa (Hybrid Search):** Superamento dei limiti semantici della similarità cosenica. Il sistema unisce il Dense Retrieval (HuggingFace/Ollama) per la comprensione del contesto, con lo Sparse Retrieval (algoritmo BM25 tramite FastEmbed) per la precisione assoluta su nomi propri, anni e identificativi fisici. Qdrant fonde i punteggi a livello di database tramite algoritmo *Reciprocal Rank Fusion (RRF)*.
- **Data Sovereignty e Offloading:** Nessun dato aziendale sensibile viene inviato a provider cloud esterni (come OpenAI o Google). Il motore generativo e i calcoli vettoriali sono delegati al server aziendale interno erogante **Ollama**.
- **Storage Persistente e Caching:** Il database vettoriale Qdrant opera su disco fisico (SSD locale). All'avvio, il sistema verifica l'esistenza dei vettori (Cache Hit): se presenti, l'indice viene caricato in millisecondi, azzerando i costi computazionali e l'overhead di rete per la re-indicizzazione.
- **Blindatura dell'Output (Pydantic):** Le risposte dell'LLM sono costrette in un rigido schema JSON (Structured Output). L'uso di Pydantic garantisce la validazione dei tipi di

dato a runtime. Qualsiasi allucinazione o attacco di Prompt Injection viene intercettato e scartato (Fail-Fast), evitando la corruzione dei sistemi a valle.

- **Event Loop Asincrono Persistente:** Il modulo `main.py` orchestra le interazioni utente e le chiamate di rete tramite un ciclo `async/await` continuo, impedendo la saturazione o chiusura anomala dei socket TCP ("Event loop is closed") e mascherando la latenza fisiologica del server remoto.

3. Manuale Operativo (Getting Started)

Il sistema è progettato per operare in stretta sinergia con l'infrastruttura di rete aziendale. Di seguito i passaggi obbligatori per l'avviamento e l'esecuzione dell'applicativo.

3.1 Requisiti Preliminari e Connessione di Rete (Obbligatorio)

Poiché l'elaborazione dell'Intelligenza Artificiale (modelli generativi ed embedding) avviene sul server locale dell'azienda e non su cloud pubblici, **è strettamente necessario essere all'interno della rete aziendale.**

Se si opera da remoto (Smart Working) o da una rete esterna, seguire questi passaggi:

1. Aprire il client **FortiClient VPN** (o client equivalente fornito dall'azienda).
2. Inserire le credenziali di accesso (User e Password) fornite dal reparto IT/Sistemisti.
3. Attendere la conferma di connessione stabilita.
4. *Nota:* Avviare lo script senza la VPN attiva causerà un errore di Timeout o Connection Refused in fase di calcolo vettoriale o inferenza.

3.2 Configurazione dell'Ambiente (.env)

Assicurarsi che nella root del progetto sia presente il file `.env` contenente le variabili di puntamento all'infrastruttura aziendale. Esempio di configurazione:

```
# URL del server aziendale locale (accessibile via VPN)
OLLAMA_URL=http://[IP_SERVER_AZIENDALE]:PORTA
# Modello LLM approvato per la generazione
OLLAMA_LLM_MODEL=gemma3:4b
# Percorso di archiviazione per il caching SSD locale
QDRANT_STORAGE_PATH=./database_vettoriale_storico
```

3.3 Avvio del Sistema

Aprire un terminale nella directory del progetto ed eseguire il comando:

```
python main.py
```

3.4 Ciclo di Vita dell'Esecuzione

Una volta avviato, il sistema seguirà un log a schermo (Plancia di Comando) indicando lo stato delle operazioni:

1. **Fase 1-3 (Ingestion & Gatekeeping):** Lettura dei file JSON dalla cartella ./dataset, validazione OOP (Duck Typing) ed estrazione dei metadati fisici (Regex).
2. **Fase 4 (Embedding):**
 - *Caso A (Primo Avvio):* Il sistema invia i chunk via VPN al server Ollama in batch da 16, calcola i vettori sparsi in locale e salva tutto nell'SSD.
 - *Caso B (Avvii Successivi):* Il sistema rileva l'SSD popolato e carica i dati istantaneamente bypassando la rete.
3. **Fase 6 (Interazione):** L'utente può digitare le proprie query a terminale. Il sistema recupererà i contesti ibridi migliori (Top-K=7), li invierà al server Ollama e, tramite Pydantic, restituirà a schermo un JSON validato, formattato e impaginato.

4. Conclusioni e Prossimi Passi

L'obiettivo di creare un motore documentale "Privacy-First" ed efficiente è stato raggiunto, risolvendo i problemi architetturali legati alla concorrenza dei file locali (File-Lock Qdrant) e alla gestione asincrona delle richieste di rete.

L'effort delle prossime settimane o mesi, sarà dedicato ad ulteriori modifiche o accorgimenti. Ma anche alla redazione dell'elaborato accademico basato su questo software.